

Date: / /

conceptually.

If dog & puppy are close in the learned embedding space, their euclidean distance could be small.

so distance can be used as similarity proxy:

smaller distance  $\rightarrow$  more similar

Cosine similarity  $\Rightarrow$  checks if vectors point in a similar direction

Euclidean distance  $\Rightarrow$  How far are these points?

### 3# What an embedding actually represents

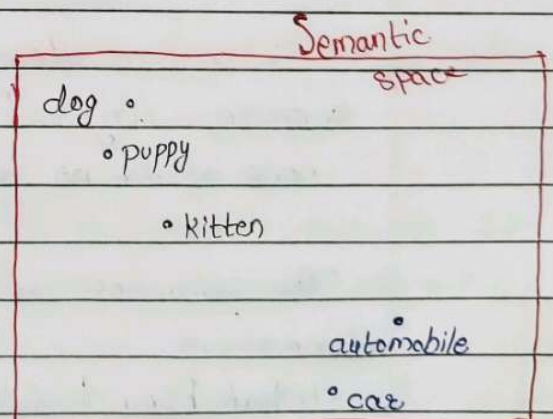
Imagine creating a map of a city.

Places with similar functions tend to appear in similar regions. (Restaurants, hotels, etc.)

Now, imagine a ML model doing something analogous, except instead of mapping physical locations, it learns to map concepts.

The model is learning a representation space in which relationships b/w concepts become mathematically useful.

This is the idea behind embeddings.



Date: / /

2. What does "representation" mean?

⇒ consider, "The puppy slept on the sofa"

A human can reason about:

puppy → animal

slept → action

sofa → furniture

& relationship b/w them.

But a NN doesn't directly operate on those human concepts. It needs numbers.

So we create a mapping:

$$f_{\theta}(x) = z \quad \text{where, } x = \text{i/p}$$

$f_{\theta}$  = learned model

$\theta$  = learned parameter

$z$  = representation ~~vector~~  
vector

for ex.

$f_{\theta}$  ("The puppy slept on the sofa")

$$= [0.12, -0.44, 0.73, \dots]$$

That vector is a representation of the input.

3. the numbers themselves are not meaningful

suppose, dog =  $[0.71, -0.14, 0.92]$

none of the no. holds any meaning i.e. they don't tell if it is animal or furry

The semantics are generally distributed across the dimensions.

What they highlight is the relationship b/w vectors.

for ex,

$$\text{dog} = [0.71, -0.14, 0.92]$$

$$\text{puppy} = [0.69, -0.11, 0.90]$$

As we can see, they are closely placed in learned representation space.

q's in short : →

- Thinking "Each dimension has a human-readable meaning" is incorrect
- Instead, "The vector as a whole participates in a learned geometry."

4) Why similar things comes close?

Here ML plays the role. The model is trained on data.

Suppose, training data repeatedly contains context like :

- The dog barked
- " puppy "
- The dog chased the ball
- " puppy " " "

So, model observes statistical regularities. Dog & puppy appears in many related contexts.

This results in fundamental principle:

similar usage/context  $\Rightarrow$  similar learned representation.

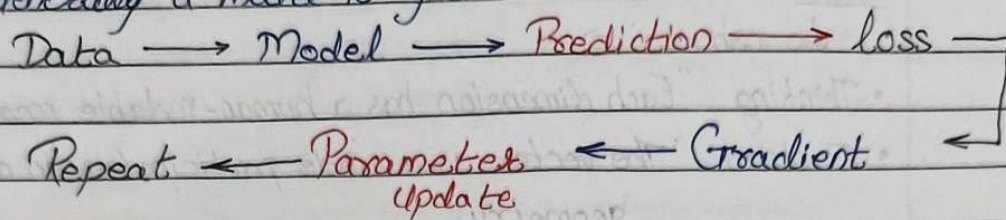
5) This is distributional learning

The learning system sees the patterns in contexts. It doesn't need a human engineer to explicitly write the relation.

Instead, the training process can discover useful structure from data.

6) Embedding space is learned.

- Generally a model is given  $\Rightarrow$



- The representation space evolves.
- The model discovers parameter that make it better at its objective.
- The resulting geometry is therefore learned.

$\Rightarrow$  Relationships can be encoded too.

Historically, some embeddings systems have exhibited approximate vector relationships like:

King - man + woman  $\approx$  Queen

So, geometries can encode relationships, not merely isolated labels.

"An embedding maps an i/p into a learned representation space where useful relationships between i/p's can be captured geometrically."

8) Word embedding vs sentence / text embedding case A)

We want "dog"  $\rightarrow$  one vector.

for ex.

$$V_{\text{dog}} = [0.2, -0.7, 0.4, \dots]$$

— This is token level representation.

Case B > Whole Sentence

We want "Dog is sleeping" → one vector.

for ex.

$$V_{\text{dog}} = [0.13, -0.44, 0.82, \dots]$$

(Sentence)

• Since a sentence contains much more info, a single word/token (e.g. Dog) cannot fully represent the diff. b/w sentences.

eg. compare "The dog is sleeping" vs.  
" " " " running"

Hence, a text embedding needs to somehow incorporate the whole ip.

9) Context matters

"The bank approved my loan" vs.

"I sat near the bank of the river"

— The word bank holds different meaning as per the context.

Date: / /

How Xmer deals with this?

The Xmer uses self-attention so that the representation of "bank" can incorporate info from surrounding tokens.  
So:

bank  $\rightarrow$  context  $\rightarrow$  contextual learning representation

Static vs contextual representation

\* Static representation

A token has one vector regardless of context

Conceptually:

bank  $\rightarrow$  vector A

\* Contextual representation

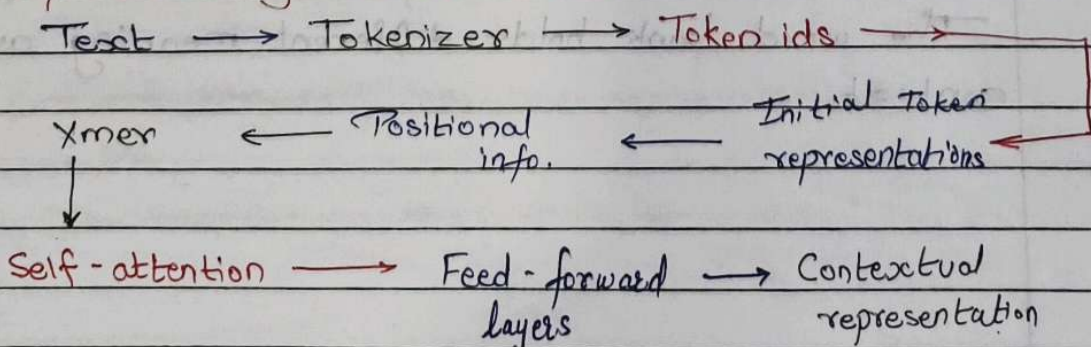
The representation changes depending on surrounding text.

"The bank approved my loan"  $\rightarrow$  Representation A

"I sat by the river bank"  $\rightarrow$  Representation B

Modern Xmer representations are generally context-sensitive

10) Connecting this to Xmers



Date: / /

The initial embedding is just a beginning. The Xmer keeps transforming the vectors.

Imagine:

"The dog chased the cat."

initially, dog  $\rightarrow$  vector  
chased  $\rightarrow$  vector  
cat  $\rightarrow$  vector

After self attention, the representation for "dog" can incorporate info about:

- chased
- cat
- sentence structure
- relationships b/w tokens

ii) Why this matters for RAG?

• Suppose company docs says:

"Employees receive 20 days of paid annual leave."

• User asks:

"How many vacation days do employees get?"

Here, the words are not exact and hence keyword search might struggle.

But a good embedding model can learn that these concepts are related.

So:  $\rightarrow$

Document  $\rightarrow$  "20 days of paid annual leave"  $\rightarrow$  Vector D

Query  $\rightarrow$  "How many vacation days?"  $\rightarrow$  Vector Q.

ideally,  $\text{sim}(Q, D)$  is high.

Date: / /

Geometrically,

they are placed in embedding ~~straight~~ space such that the model identifies the two to be identical.

That's the fundamental reason embeddings are useful in RAG.

### # Understanding whole chain

#### Step 1: Tokenizer

Text  $\longrightarrow$  tokens  $\longrightarrow$  token IDs

#### Step 2: Model process the token representations.

token representations  $\longrightarrow$  Transformer  $\longrightarrow$  Contextual understanding

#### Step 3:

Embedding systems creates:

$$q \in \mathbb{R}^d$$

$q \rightarrow$  query vector.

Now we have document chunks:

Chunk 1  $\longrightarrow$  vector 1  $d_1$

Chunk 2  $\longrightarrow$  vector 2  $d_2$

Chunk 3  $\longrightarrow$  vector 3  $d_3$

The system compares  $q$  against  $d_1, d_2, d_3, \dots$  using something like cosine similarity.

Then,

highest similarity  $\longrightarrow$  most promising chunk  $\longrightarrow$   
LLM  $\longleftarrow$  RAG context  $\longleftarrow$

Date: / /

do

